Lecture 13: Advanced Training Workflows: experiment tracking (W&B), multi-GPU, serverless training (Vertex AI)

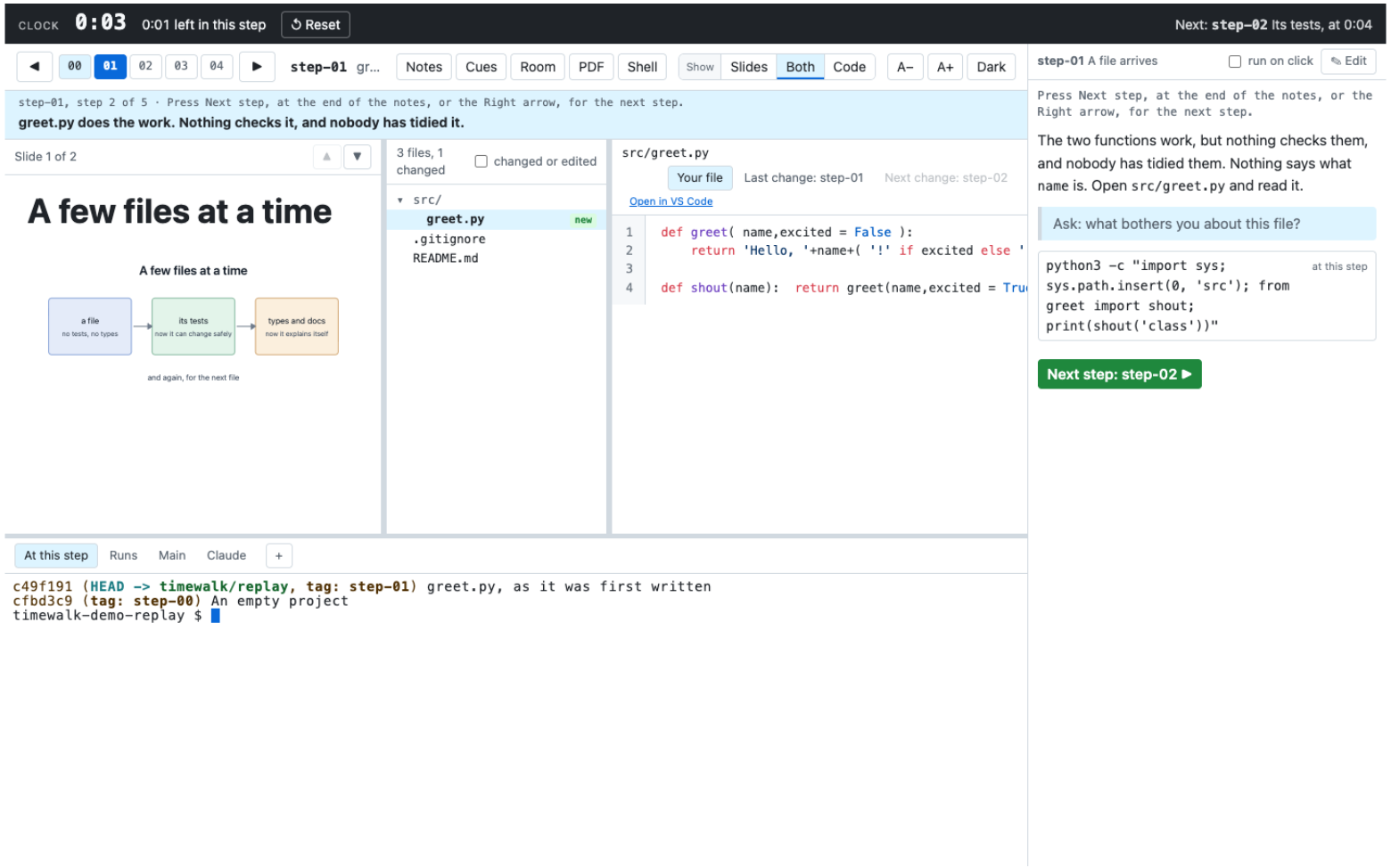
AC215/E115

Pavlos Protopapas and Rahul Dave
SEAS/Harvard



Timewalk: a project at each step

Code, slides and terminals follow the same tagged history.



A step is a named point in the project history.

The page shows the files and changes at that step.

Run its commands in the local terminal.

rahuldave.com/timewalk

GitHub: rahuldave/timewalk

Timewalk: watch a move or make it yourself

Watch mode

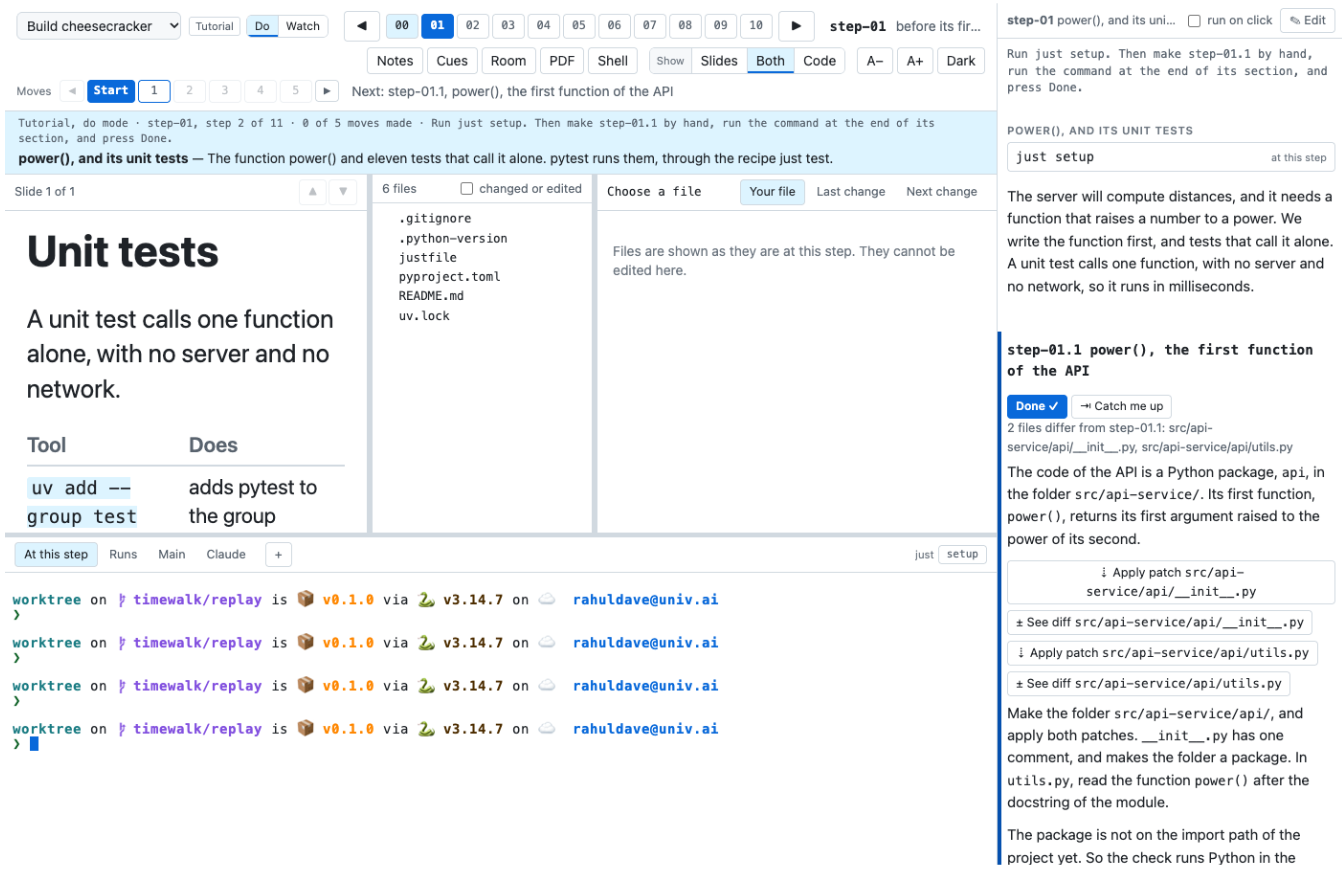
Timewalk checks out each move.
Read the change and try its checks.

Do mode

You edit the files or apply the patch.
Done checks them against the move.

Fork a walk, clone your fork, then run:

```
just present
```



Prerequisites: uv, git, just and a recent browser.

The technology: commits, steps and walk notes

Git records the code. The walk explains how to follow it.

Unit	What it records	How a learner uses it
Move	One small commit	Inspect one change, or make it by hand.
Step	A tag at a working checkpoint	Finish a part with its checks passing.
Walk	Notes linked to that history	Read the explanations or follow them in Timewalk.

The project repository holds the code and history.
The public walk repository holds the student notes and recipes.

Tiny commits make up one step

Cheesecracker step 01: power() and its unit tests

Move	One change	Commit
01.1	Add power()	bb1701f
01.2	Add pytest to the test group	fc8c3bc
01.3	Set test discovery and imports (+5 lines)	ff82ac1
01.4	Add eleven unit tests	cbc98b1
01.5	Add the just test recipe (+4 lines)	e86e0c3

The step-01 tag names the final move.
Small changes let us see exactly what each move adds.

GitHub: the five commits from step-00 to step-01

First walk: cheesecracker

A small API server and its checks

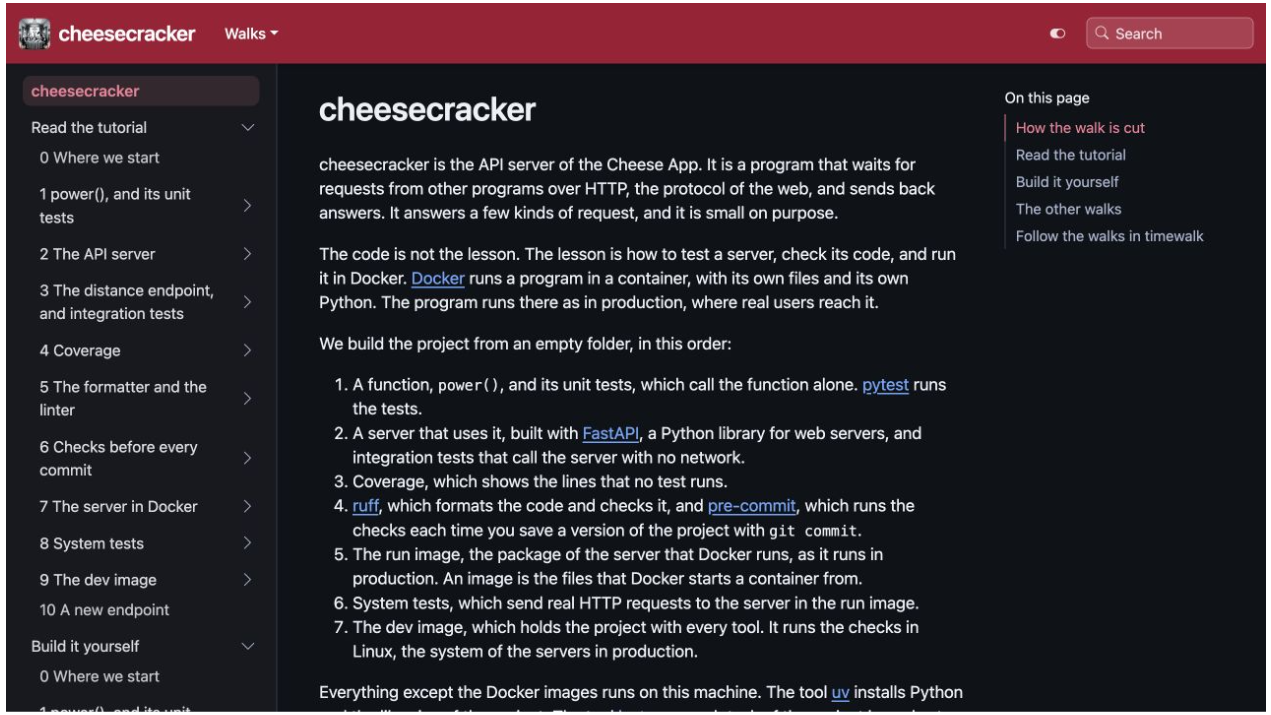
Start with power() and unit tests.
Then test a FastAPI server.
Finish with checks and Docker.

Walk repository

github.com/rahuldave/cheesecracker-walk

Project repository

github.com/rahuldave/cheesecracker



Read the tutorial or Build it yourself.

Website: ac215-testing.dlops.io/cheesecracker-walk/

Second walk: smallwikiapp

A command becomes a Python project

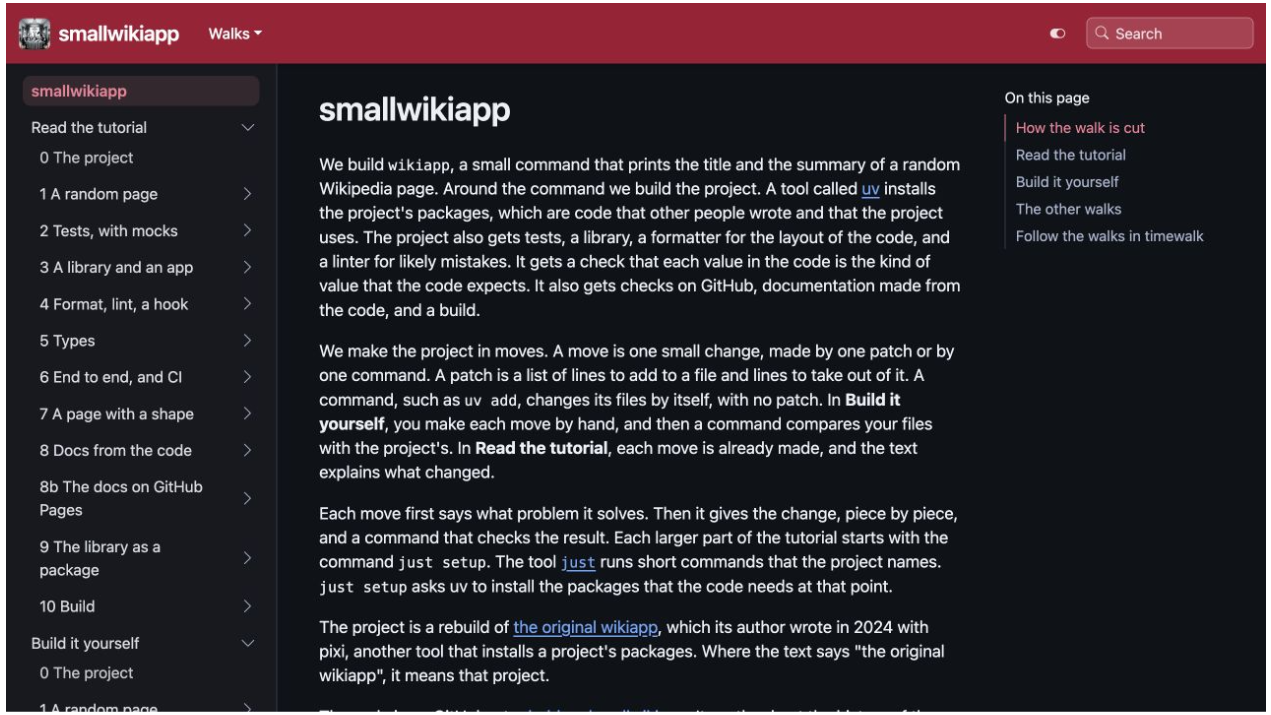
Fetch a random Wikipedia summary.
Add tests with mocks and a library.
Then add types, CI, docs and a build.

Walk repository

github.com/rahuldave/smallwikiapp-walk

Project repository

github.com/rahuldave/smallwikiapp



Read the tutorial or Build it yourself.

Website: ac215-testing.dlops.io/smallwikiapp-walk/

Third walk: babykev

Tests around a decision model

Define questions and typed answers.
Test the contract and model.
Inspect masks with a tiny model.

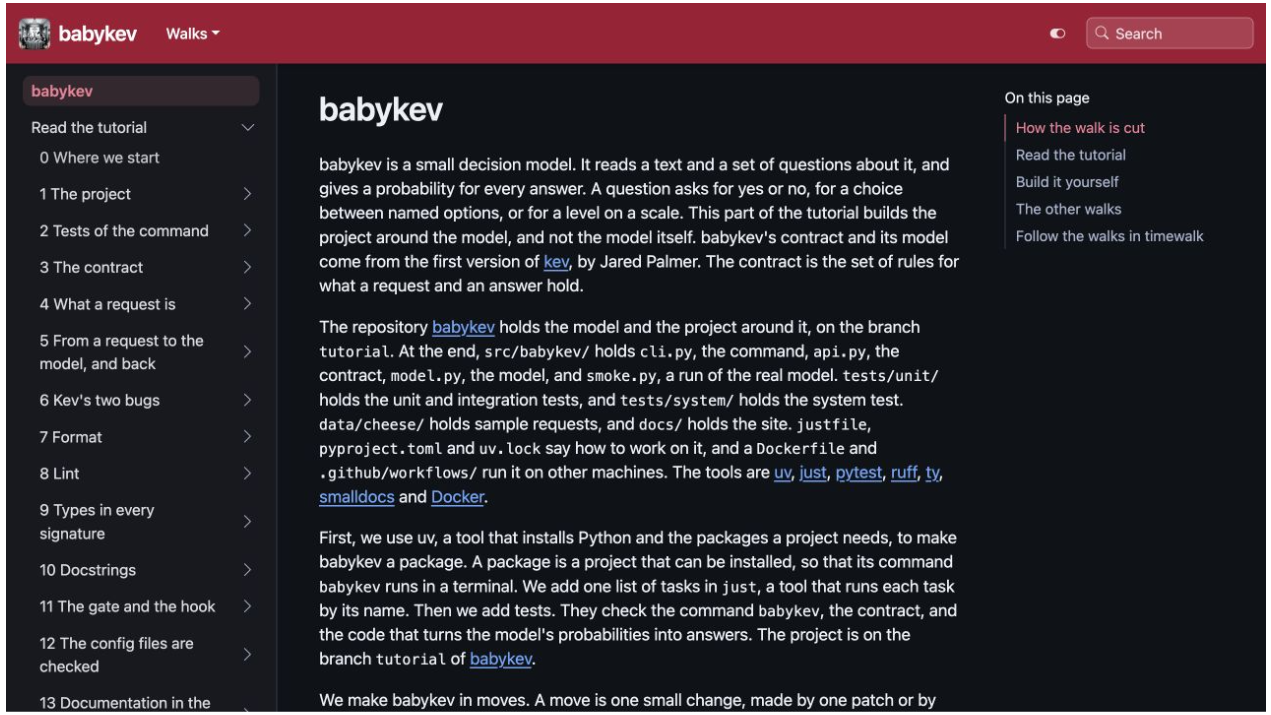
Walk repository

github.com/rahuldave/babykev-walk

Project repository

github.com/rahuldave/babykev/tree/tutorial

Website: ac215-testing.dlops.io/babykev-walk/



Read the tutorial or Build it yourself.

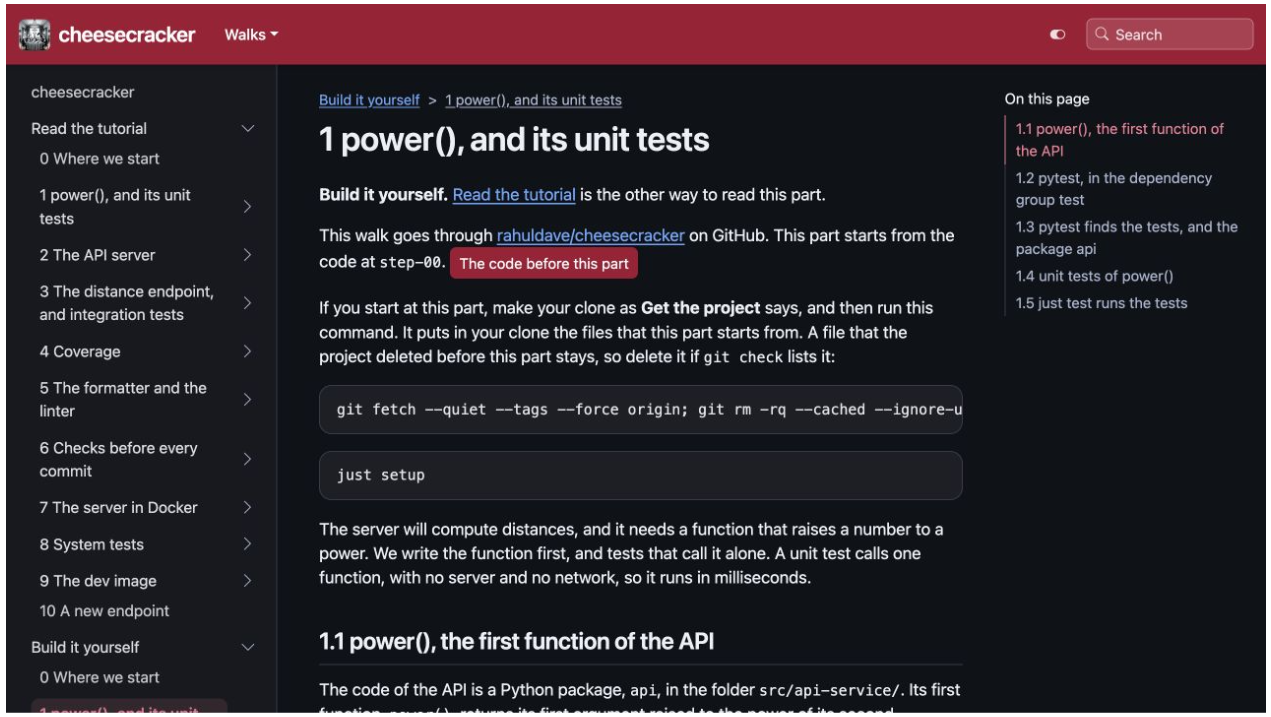
Cheesecracker do mode: Build it yourself

You make each move in your local clone.

Start from the page's stated commit.
Run each command or apply its patch.

Run the check after each move.
Compare your files with git check.

The website supplies instructions.
You work locally with Git.



Open the Build it yourself page

The website route works without running Timewalk.

Cheesecracker walk mode: Read the tutorial

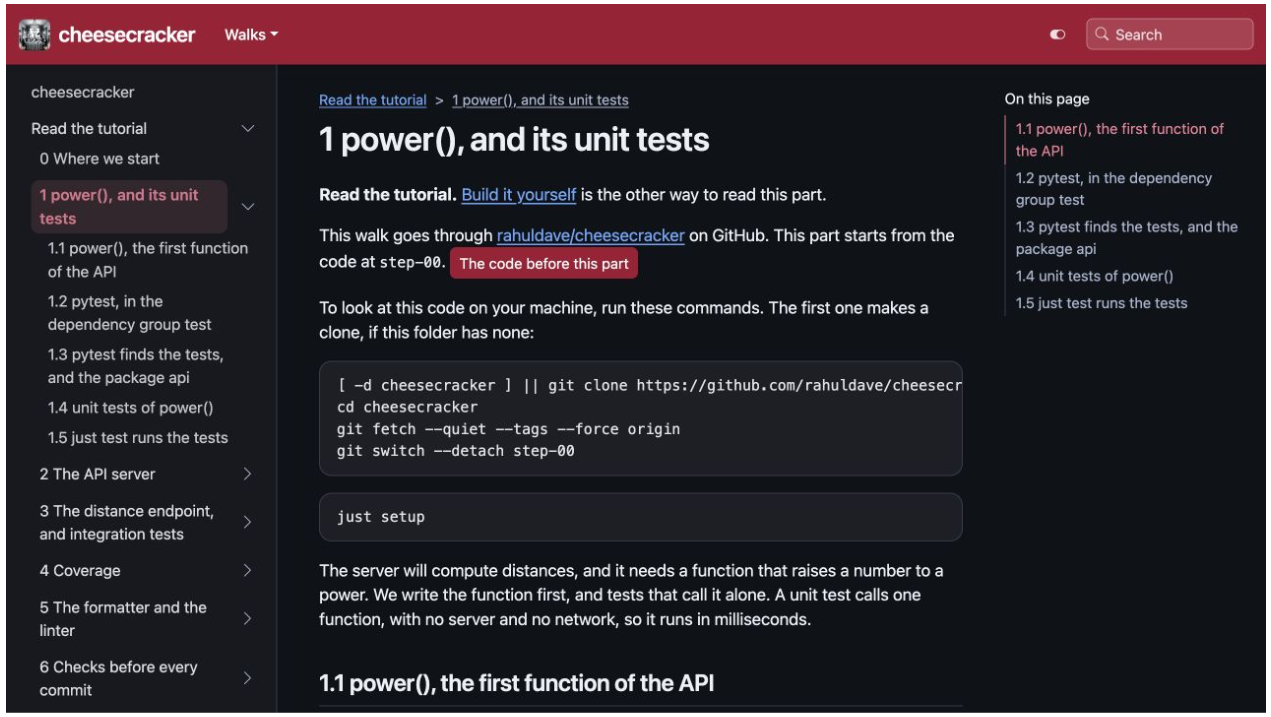
Read the recorded changes and inspect the code.

Each move explains a small change.
Follow links to the code on GitHub.

What changed lists the files
and their diffs for that part.

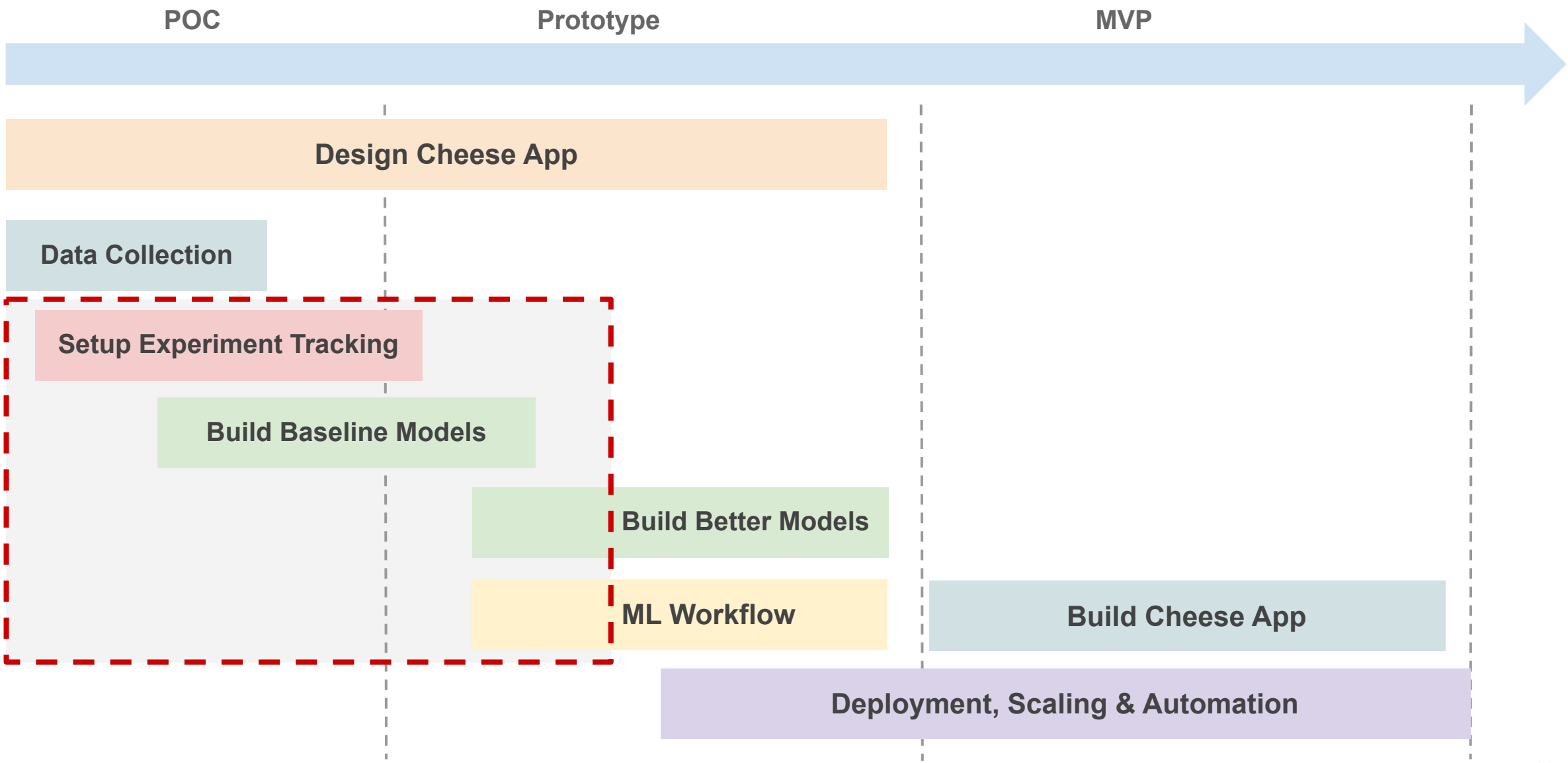
Use the same step order as do mode.
Switch to Build it yourself to practise.

The website route works without running Timewalk.



Open the Read the tutorial page

Review: Cheese App Status



Recap: Motivation

The 3 components for better Deep Learning



More Data

- Extraction
- Transformation
- Labeling
- Versioning
- Storage

- Processing
- Input to Training



Better/Faster Models

- SOTA Models
- Transfer Learning
- Distillation
- Compression



Faster Hardware

- Scaling data processing
- GPU, TPU
- Multi GPU Server Training

Outline

1. **Models we are looking at**
2. Testing model behavior
3. Experiment tracking with W&B
4. Vertex AI / Agent Platform training
5. Parallel training

MNIST: three models for handwritten digits

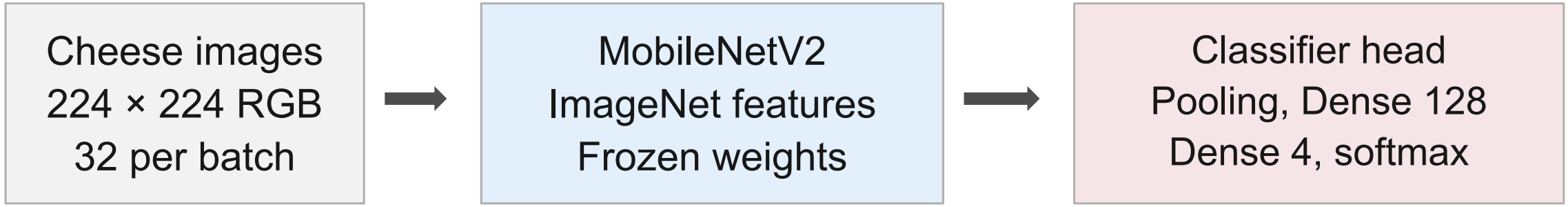
Input: one 28 × 28 grayscale image. Output: scores for digits 0–9.

Model	How it processes the image
Logistic regression	Flatten 784 pixels. One linear layer gives 10 scores.
MLP	Flatten pixels. A hidden layer with 128 units learns features.
ConvNet	Two convolution + pooling stages learn local image patterns. A dense classifier gives the final 10 scores.

All weights train from scratch. Cross-entropy consumes the ten raw scores (logits).
We will track one run per model in W&B using the same split and training recipe.

The cheese image classifier

Reuse the image backbone. Learn the cheese classifier head.



Prediction: compute four probabilities and choose the largest.

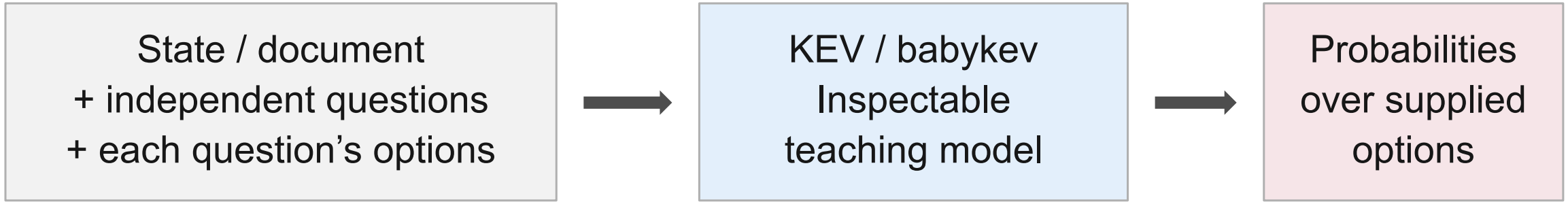
Learning: SGD updates the head. Backbone weights stay fixed.

Training mode: frozen MobileNetV2 + trainable head. No LoRA adapters.
Classes: brie, gouda, gruyere, parmigiano. Global batch 32. SGD learning rate 0.001.

KEV: a decision model

A decision model answers supplied questions about a shared state.

Jev, from TypeSafe, makes typed decisions from unstructured context.
Its Boolean, choice and ordered-score questions are independent.



Course KEV freezes Qwen2.5. We train small LoRA adapter layers and the pointer head.
In evaluation, changing one question must not change another question's answer.

Question types: options in, typed answers out

The API validates the type, then builds one option list per question

Type	Options / toy probabilities	Readout	Answer
Boolean (noul)	no: 0.2, yes: 0.8	Read p(yes)	0.8
Choice	cow: 0.7, sheep: 0.2 goat: 0.1	Name at argmax	cow
Ordered score	mild: 0.1, medium: 0.2 strong: 0.7	0×0.1 + 1×0.2 + 2×0.7	1.6

Toy probabilities. One shared pointer head scores every option list.
Score levels use positions 0, 1, 2. The API can return a fractional expected index.

The type stays in API metadata

The model reads instructions and options. The API saves the answer type.

Model-bound text excerpt

```
instr: "Is it soft?"
options: ["no", "yes"]
```

Saved outside the model input

```
id: "soft"
type: "noul"
```

STATE
Brie is soft…

→

QUESTION Is it soft?
OPTION no END
OPTION yes END
DECIDE

Mnemonic marker names shown. All types use the same five delimiter roles.
Type/id stay in metadata. Option names and descriptions reach Qwen as ordinary text.

Independent questions need an extra mask rule

Shared document: “Brie is soft and made from cow’s milk.”
A asks “Is it soft?” B asks “Which milk?” Each answers separately.

Causal only: B can read A

Row: reader / Column: token being read

	D1	D2	A1	A2	B1	B2
D1	1	0	0	0	0	0
D2	1	1	0	0	0	0
A1	1	1	1	0	0	0
A2	1	1	1	1	0	0
B1	1	1	**1**	**1**	1	0
B2	1	1	**1**	**1**	1	1

Causal + boundary: B cannot read A

Row: reader / Column: token being read

	D1	D2	A1	A2	B1	B2
D1	1	0	0	0	0	0
D2	1	1	0	0	0	0
A1	1	1	1	0	0	0
A2	1	1	1	1	0	0
B1	1	1	**0**	**0**	1	0
B2	1	1	**0**	**0**	1	1

1 = may read. 0 = blocked. D1 and D2 are tokens of one shared document.
A1/A2 and B1/B2 belong to separate question branches. Editing A must not change B in evaluation.

Outline

1. Models we are looking at
2. **Testing model behavior**
3. Experiment tracking with W&B
4. Vertex AI / Agent Platform training
5. Parallel training

pytest and assertions

A unit test calls one function and checks its result

```
def test_choice_confidence_is_zero_for_a_guess():
    p = [0.25, 0.25, 0.25, 0.25]
    assert choice_confidence(p) == 0.0
```

The assertion states the contract: a uniform guess has zero confidence.

Historical step 01: 52 cases, 50 passed, 2 failed, 100% coverage.
Coverage records execution. An assertion checks behavior.

Parametrization: one test, several inputs

Each pair supplies an input and the answer we expect.

```
@pytest.mark.parametrize(
    ("value", "expected"),
    [(0, 0), (-3, 3), (5, 5)],
)
def test_absolute_value(value, expected):
    assert abs(value) == expected
```

pytest runs three cases. A failure identifies the input that failed.

Requires import pytest. The test body stays the same while the inputs change.

Edge cases that found real bugs

Uniform guess over three levels

```
assert score_confidence([1 / 3] * 3) == pytest.approx(0)
```

Historical result: 0.5. The contract expects zero confidence.

Rounding 77 equal probabilities

77 × 0.01 = 0.77

The displayed probabilities no longer sum to one.

Both failures came from edge cases in step 01 of the source walk.

Fixtures: database setup and teardown

A fixture prepares a resource for a test and cleans it up afterward.

Fixture: bring up, yield, tear down

```
@pytest.fixture
def db():
    conn = sqlite3.connect(":memory:")
    try:
        conn.execute("CREATE TABLE users (id INT)")
        yield conn
    finally:
        conn.close()
```

Test: ask for db by name

```
def test_empty_database(db):
    rows = db.execute(
        "SELECT * FROM users"
    ).fetchall()
    assert rows == []
```

SETUP: open the database and create the table before the test.
TEARDOWN: finally closes the database, even if the test fails.

Requires import sqlite3 and import pytest. yield gives the connection to the test.
Each requesting test gets a fresh in-memory database. Closing it discards its contents.

tiny_tokenizer(): a tokenizer for fast tests

A tokenizer turns text into IDs. This one splits ordinary text into characters.

Pseudocode: ordinary builder function

```
def tiny_tokenizer():
    vocab = {"[UNK]": 0}
    add_printable_character_ids(vocab)
    tok = character_tokenizer(vocab)
    tok.add_special_tokens(SPECIAL)
    return tok
```

Example

Text: "cat"
Tokens: [c, a, t]

Built in memory.
No download.

SPECIAL = five delimiters.
[UNK] = unknown character.

Like db() calling sqlite3.connect(), the tok() fixture calls this builder.
The helper is ordinary Python. The fixture decides to share one tokenizer per test session.

tiny_backbone(): a tiny Qwen decoder

Use Qwen's decoder implementation with a very small configuration.

Pseudocode: ordinary builder function

```
def tiny_backbone(vocab_size, seed=0):
    torch.manual_seed(seed)
    config = Qwen2Config(
        vocab_size=vocab_size,
        hidden_size=32,
        intermediate_size=64,
        num_hidden_layers=2,
        num_attention_heads=2,
        num_key_value_heads=2,
    )
    return Qwen2Model(config)
```

Two decoder layers
32-number hidden vectors

Random initial weights.
No trained model
download.

Like db() building a connection, model(tok) builds a fresh CPU model for each test.
Tests exercise packing, masking and the pointer head. Random weights do not establish answer quality.

Model fixtures use the same pattern as the database

pytest builds the named resource, then passes it to a test that asks for it.

```
@pytest.fixture(scope="session")
def tok():
    return tiny_tokenizer()

@pytest.fixture
def model(tok):
    backbone = tiny_backbone(len(tok))
    return DecisionModel("tiny", "cpu",
                         backbone=backbone).eval()
```

db → database connection tok → tokenizer model → KEV test model

tok: once per session. model: fresh per test. The builders themselves are not fixtures. Database teardown calls close(). These CPU stand-ins need no external service teardown.

Kinds of tests

Kind	Scope in this project	Purpose
Unit	One confidence function	Check a local rule
Integration	Packing + tiny model + head	Check parts together
System	Program with actual Qwen	Check the actual stack
Smoke	Short system run	Check startup and output
Acceptance	A stated user requirement	Check a promised outcome

Teams use these names differently. State which components and dependencies a test uses.
A smoke test can also be a system test.

Properties of independent questions

Sibling isolation

Changing one question preserves its siblings' scores.

Packed versus separate

Packing several questions preserves each answer's scores.

Question order

Reordering questions preserves their corresponding scores.

Causality

No token reads a later token.

The tiny model can check these rules with controlled weights and evaluation mode.

The mask bug and its code fix

The old boundary condition was always true, so only the causal rule remained.

BEFORE: every token pair passes the boundary check

```
# All entries start True. OR cannot block any token pair.
same = torch.ones_like(causal) | (s[None, :] == 0)
```

AFTER: shared document or same question

```
# s[:, None] gives reader IDs; s[None, :] gives source IDs.
# Allow the same question, or a source in the shared document (0).
same = (s[None, :] == s[:, None]) | (s[None, :] == 0)
# AND blocks future tokens and other questions.
allow = causal & same
```

causal blocks future packed tokens. same blocks tokens from a different question branch.
s stores segment IDs: 0 is the shared document; 1, 2, … identify the question branches.

Why the next test is an integration test

It checks several cooperating parts through the model's public call path.

Unit test: one part

branch_mask(seg) alone: does this matrix block the forbidden links?

Integration test: the parts together

encode packs the request and assigns segment/position IDs.
DecisionModel passes its mask into the tiny Qwen decoder.
The pointer head turns decoder outputs into option scores.

Changing B must preserve the final scores for A and C across that whole path.
The backbone is tiny and randomly initialized. The packing, mask and pointer head are real code.

Integration test: edit B, keep A and C unchanged

Use the same tiny model twice. Change only B's question and options.

A and C stay fixed

A: "Is it blue?" (2 options)
C: "How firm?" (3 options)

Only B changes

"Which milk?" (3 options)
becomes "Where from?" (2 options)

```
before = logits_of(model, tok, record)
changed = deepcopy(record)
changed["questions"][1] = {"instr": "Where from?",
                           "options": ["Spain", "Italy"], "label": 0}
after = logits_of(model, tok, changed)
assert torch.equal(before[0], after[0])  # A unchanged
assert torch.equal(before[2], after[2])  # C unchanged
assert len(before[1]) == 3 and len(after[1]) == 2
```

model, tok and record are fixtures. logits_of packs the request and returns option-score vectors.
The test checks unchanged scores in evaluation mode. It does not check whether the answers are correct.

Checks without external services

Local deterministic checks

Confidence rules and attention masks
Packing, the tiny model, and the head
No Hub download or external service

Checks with external dependencies

Actual Qwen weights, tokenizer, and GPU environment
A separate run records the model revision and hardware.

A marker selects a dependency group. The test name alone does not describe its scope.

Checks beyond pytest

Target	Tool or check	Failure it can detect
Behavior	pytest + Hypothesis	Broken invariants
Types	ty	Invalid type assumptions
Documentation	Documentation checker	Missing or stale API entries
Format + lint	ruff	Formatting and static findings
Configuration	YAML, package, container checks	Invalid build or runtime inputs

Different checks catch different defects. They can share one local entry point.

One check command

The source project puts its checks behind one recipe

```
# Source project: step 03, just check
just fmt --check
just lint
just test -q --cov-fail-under=90
just lint-config
```

Local checks and the commit hook call the same recipe.

A passing check only establishes the behavior its assertions and rules cover.

Outline

1. Models we are looking at
2. Testing model behavior
3. **Experiment tracking with W&B**
4. Vertex AI / Agent Platform training
5. Parallel training

Experiment Tracking – Why It Matters

Organization: Keep your work structured — data collection, model training, and results.

Reproducibility: Be able to repeat experiments and trust the outcomes.

Logging: Record metrics, parameters, and outcomes automatically.

Collaboration: Make it easier for teammates to follow and build on your work.

Accountability: Know exactly what led to a model's performance — good or bad.

Experiment Tracking - What to Track

- **Environment:** Python version, dependencies, and hardware setup
- **Code:** Scripts and configurations used for training
- **Data:** Version, splits (train/validation/test), and preprocessing steps
- **Model:** Architecture, hyperparameters, and configurations
- **Metrics:** Evaluation metrics and performance results
- **Artifacts:** Model weights, logs, and sample predictions
- **Notes:** Any observations or experiment context

Where do we track what?

Item	Where it is tracked
Code and scripts	Git repository
Data and versions	DVC, lakeFS, Hugging Face Datasets Record the exact revision and split.
Environment and config	pyproject.toml and uv.lock in Git
Model weights	Hugging Face model repository; W&B Artifacts
Metrics and logs	W&B run history and cloud-job logs

More on data versioning later. A storage bucket alone does not freeze a dataset.
Link the data/model revisions to the experiment run.

Hugging Face: the Hub and its ecosystem

What can we use?

Models
Weights, config, tokenizer and a model card.

Datasets
Data files, splits and a dataset card.

Spaces
Hosted interactive demos.

Tools and services
Transformers, Datasets, Hub client;
hosted inference and Jobs.

Large files: Git LFS to Xet

Git keeps version history and small pointers to large binary files.

Historically: Git LFS stored the large payload separately.

Now: Xet stores reusable chunks.
Shared chunks need not be sent again.
Git LFS compatibility remains.

Pin the dataset and model commit revisions. Libraries load the selected files into your program.
Hub storage is separate from the GPU memory used to train or run a model.

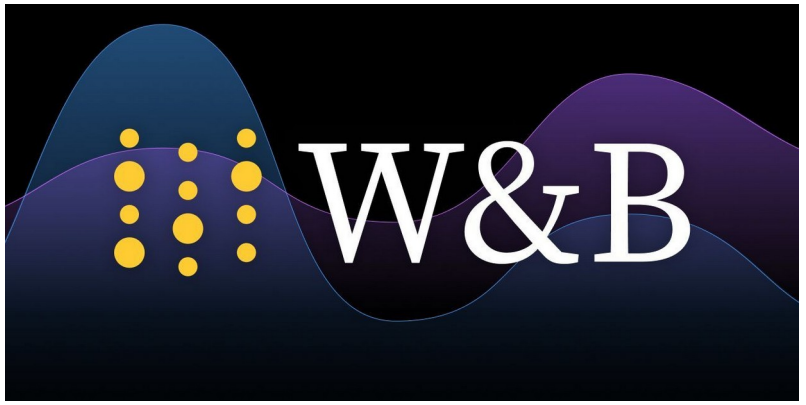
Experiment Tracking Tools

Tool	Key Features	Common Use Case
Weights & Biases (W&B)	Cloud-based, easy UI, integrates with most ML frameworks	Research, production ML tracking
MLflow	Open-source, self-hosted option	Enterprise or private infrastructure
Comet	Experiment comparison, team dashboards	Collaboration-focused workflows
Neptune	Metadata storage, flexible APIs	Custom ML pipelines

Bottom note: *We'll use **Weights & Biases** today because it's simple, visual, and integrates seamlessly with Keras/Tensorflow and PyTorch.*

What is Weights & Biases (W&B)?

- A **platform for tracking, visualizing, and managing** machine learning experiments
- Integrates easily with frameworks like **TensorFlow**, **PyTorch**, and **Scikit-learn**
- Automatically logs:
  - Hyperparameters
  - Metrics
  - Model versions
  - System performance
- Includes project dashboards, reports, and collaboration tools
- Free tier available for education and research



Colab: an interactive notebook with a GPU

Run the MNIST comparison in the shared Colab notebook

1. Open the notebook and save your own copy.

2. Select a GPU runtime when one is available.

3. Run the setup, train the models, then inspect the W&B runs.

Open the public MNIST Colab

The notebook is world-viewable. Your runtime and W&B account are separate.
Our saved comparison used one T4. A new Colab session may allocate different resources.

MNIST: logging an epoch to W&B

One run stores the recipe, learning history and final test summary.

```
run = wandb.init(project=project, config=recipe)
run.define_metric("epoch")
run.define_metric("validation/*", step_metric="epoch")

run.log({"epoch": epoch,
         "validation/accuracy": val_accuracy})
run.summary["test/accuracy"] = test_accuracy
```

Stylized excerpt. The notebook also logs train/loss, train/accuracy and validation/loss.
Use validation during development. Read test/accuracy from the final summary.

MNIST: three models tracked in W&B

Same split and recipe. One W&B run per model.

Completed Colab quick comparison, October 17, 2026. One Tesla T4 GPU.

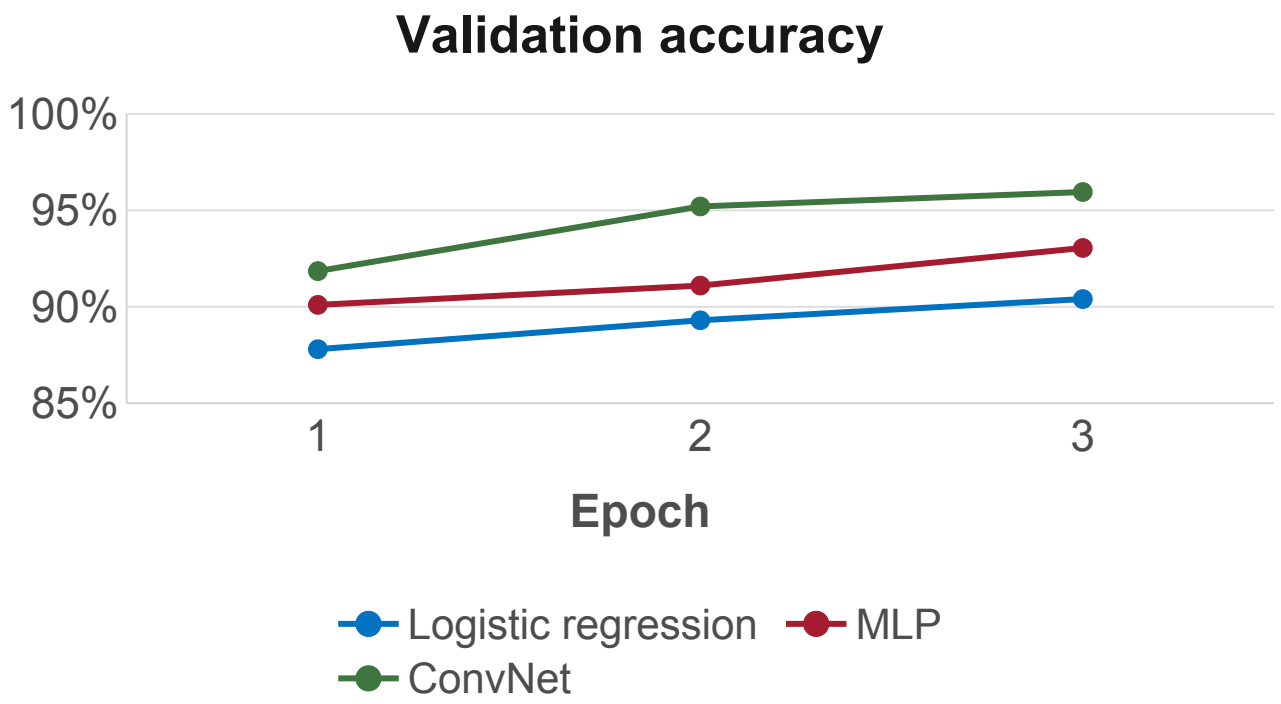
Model	Parameters	Validation accuracy	Test accuracy
Logistic regression	7,850	90.40%	90.71%
MLP	101,770	93.05%	92.91%
ConvNet	206,922	95.95%	96.79%

Logistic regression run MLP run ConvNet run

10,000 train / 2,000 validation / 10,000 test. Three epochs per model.
Adam, learning rate 0.001, batch 128, seed 2026. Test uses final-epoch weights.

MNIST: comparing the runs in W&B

Compare learning curves, then read final held-out scores



Open W&B comparison

Training history
epoch versus validation/accuracy
One curve per model.

Final scores: test/accuracy
Tune using validation, then test.

Open the Colab notebook

Curves reproduce saved Colab validation history. Links require W&B and notebook access.
One seed and a shared recipe. These runs do not establish each model's best accuracy.

Outline

1. Models we are looking at
2. Testing model behavior
3. Experiment tracking with W&B
4. **Vertex AI / Agent Platform training**
5. Parallel training

Vertex AI and its new name

Name in the original course and tutorial

Vertex AI

Current platform name

Gemini Enterprise Agent Platform

The console also uses the shorter name Agent Platform.

We keep both names. Managed Training / Serverless Training runs our own training program.

Training pages: Vertex AI / Agent Platform

Select your Google Cloud project and the training region first

View submitted training runs

Agent Platform: Custom jobs

Create through the console wizard

Training pipelines → Create → Custom training (advanced)

The wizard creates a training pipeline that starts a custom job.

CLI and Python SDK jobs appear under Custom jobs. The slide links open the current routes.

Serverless training: request resources for a job

Ask the platform to run your training program on chosen hardware

1. Package the code and choose its runtime.

2. For GPUs: request regional training quota, then check approval later.

3. Submit the job once quota is effective. The service runs the workers.

A request for 2 GPUs does not grant quota. Check the same request again after review. Confirm the effective regional limit. Quota approval does not guarantee available capacity.

Serverless training: 1. move code into a module

A notebook is interactive. A job needs a program it can start and finish.

package/trainer/task.py, condensed

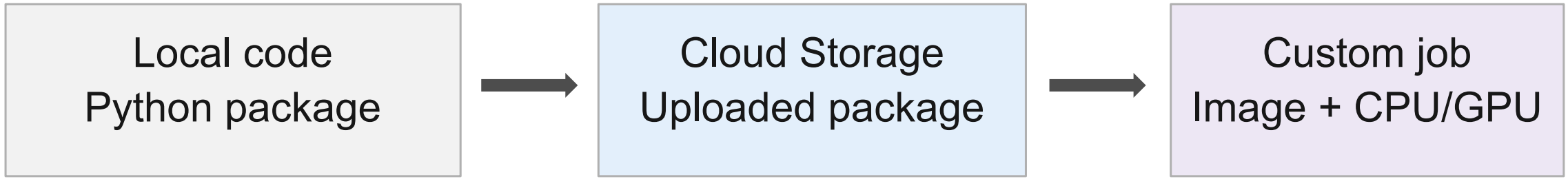
```
def main(args):
    train, validation = load_cheese_data(args)
    model = build_mobilenet_model(train_base=False)
    model.fit(train, validation_data=validation,
              epochs=args.epochs)

if __name__ == "__main__":
    main(parse_args())
```

The job starts trainer.task with command-line arguments. Helpers/imports are omitted here.
Next: 2. package and upload; 3. create and run. The completed cheese recipe uses one epoch.

Serverless training: 2. package; 3. create and run

Step 2 uploads the trainer. Step 3 starts it in a compatible training image.



Cheese example: package-trainer.sh builds the archive; cli.sh or cli.py submits it.

The worked cheese commands follow: package/ contains setup.py and trainer/.
Vertex installs that uploaded package. Your code decides which results and weights it saves.

Cheese tutorial: Vertex AI / Agent Platform

Use the existing tutorial's package and launch scripts

1. Confirm project, region, permissions, bucket, and GPU quota.

2. Package and upload the Python trainer.

3. Choose the image and hardware, then submit the job.

Open the tutorial

Open Custom jobs

The scripts use gcloud ai and google.cloud.aiplatform. Their names still contain the older API terms.
W&B displays the training metrics; the cloud console displays job execution.

The cheese worker: an N1 host plus T4 GPUs

N1 names the CPU/RAM machine family. T4 names an attached GPU.

Our host: n1-standard-4

One virtual machine with
4 virtual CPUs and 15 GB RAM.
It loads data and runs Python.

Attached accelerators: NVIDIA T4

One or two GPUs, each with
16 GB of device memory.
TensorFlow places model work here.

n1-standard-4 + 2 T4s is still one VM, with two GPUs. Regional quota was approved at 2.
GPU memory is separate from host RAM. The final section explains the two-GPU computation.

Cheese training: the single-GPU Python program

One T4: run trainer.task once. TensorFlow sends tensor operations to that GPU.

1. Load batches and build the model

```
datasets = get_dataset(batch_size=32)
train_data, val_data, _ = datasets
model = build_mobilenet_model(
    image_height, image_width,
    num_channels, num_classes,
    model_name, train_base=False)
logger = WandbMetricsLogger(
    log_freq="epoch")
```

Frozen MobileNetV2 + trainable head.
Global batch 32; final batch 8.

2. Configure training, then fit

```
optimizer = keras.optimizers.SGD(
    learning_rate=args.lr)
model.compile(optimizer=optimizer,
    loss=loss, metrics=[],
    jit_compile=False)
model.fit(train_data,
    validation_data=val_data,
    epochs=args.epochs, shuffle=False,
    callbacks=[logger])
```

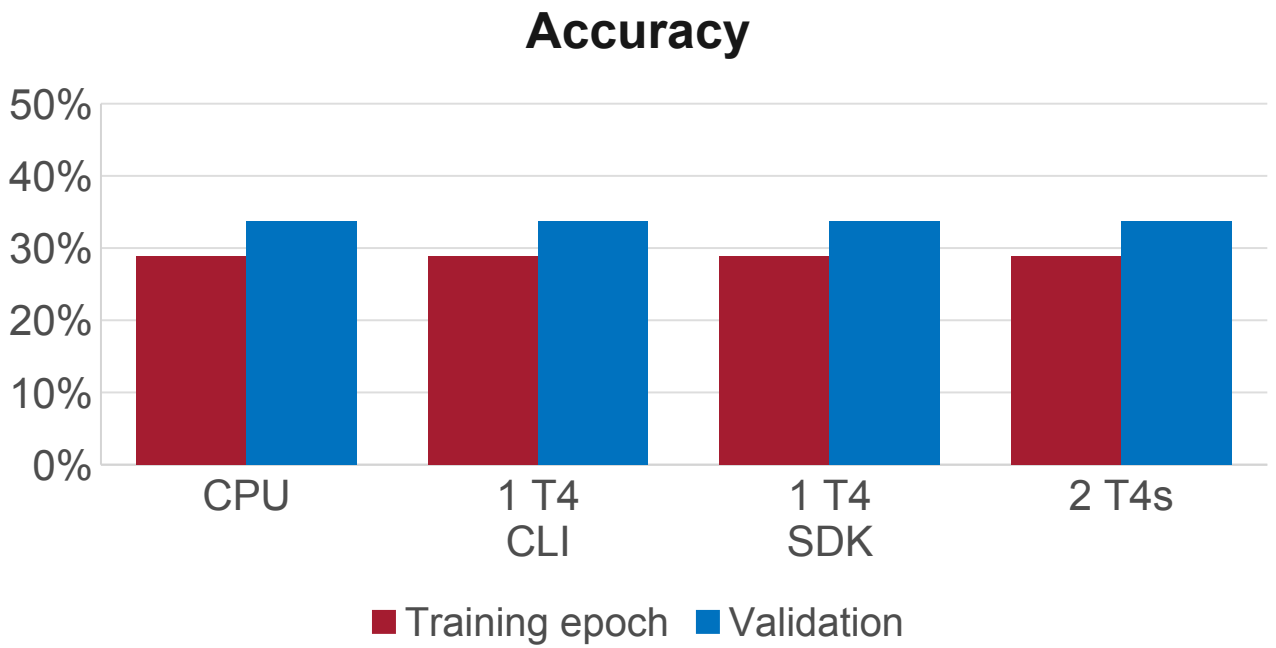
fit runs the batch loop.
The callback logs each epoch to W&B.

Condensed source. loss is categorical crossentropy; our model helper normalizes loss and metrics.
The parallel section adds MirroredStrategy for two T4s: 16 images per GPU, global batch still 32.

Cheese: corrected epoch metrics

Every route: 28.85% training, 33.65% validation

1 epoch; SGD 0.001; global batch 32; seed 42; 30 updates; frozen MobileNetV2 + head



Loss includes L1

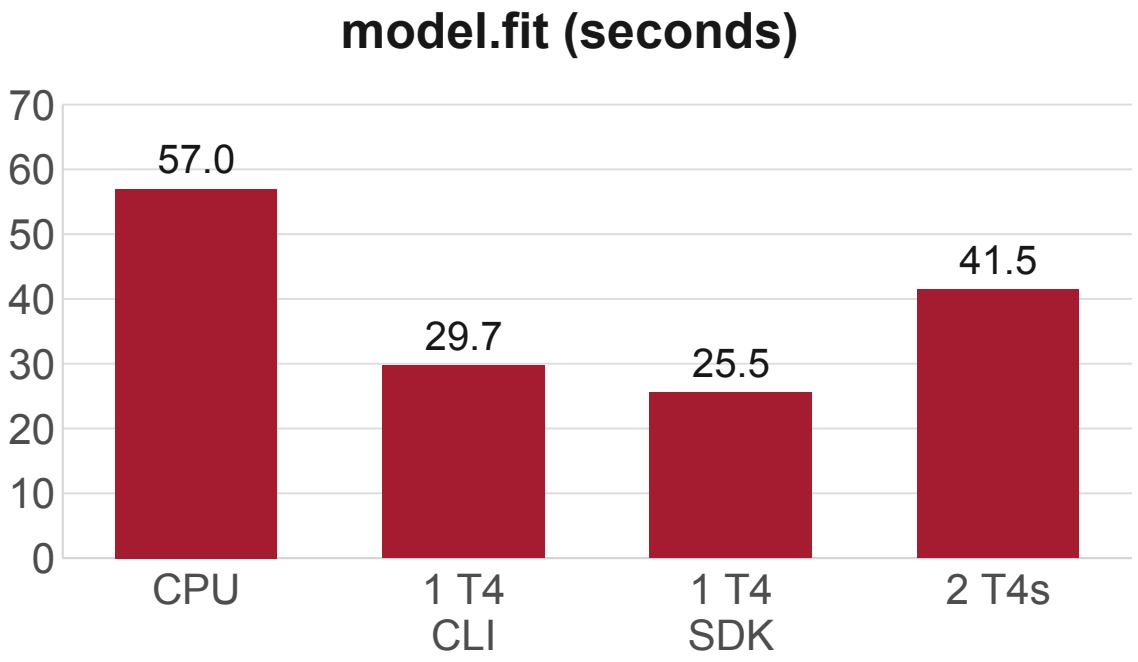
Route	Train loss	Val. loss
CPU	108.49	107.36
1 T4 CLI	108.49	107.36
1 T4 SDK	108.49	107.36
2 T4s	108.49	107.36

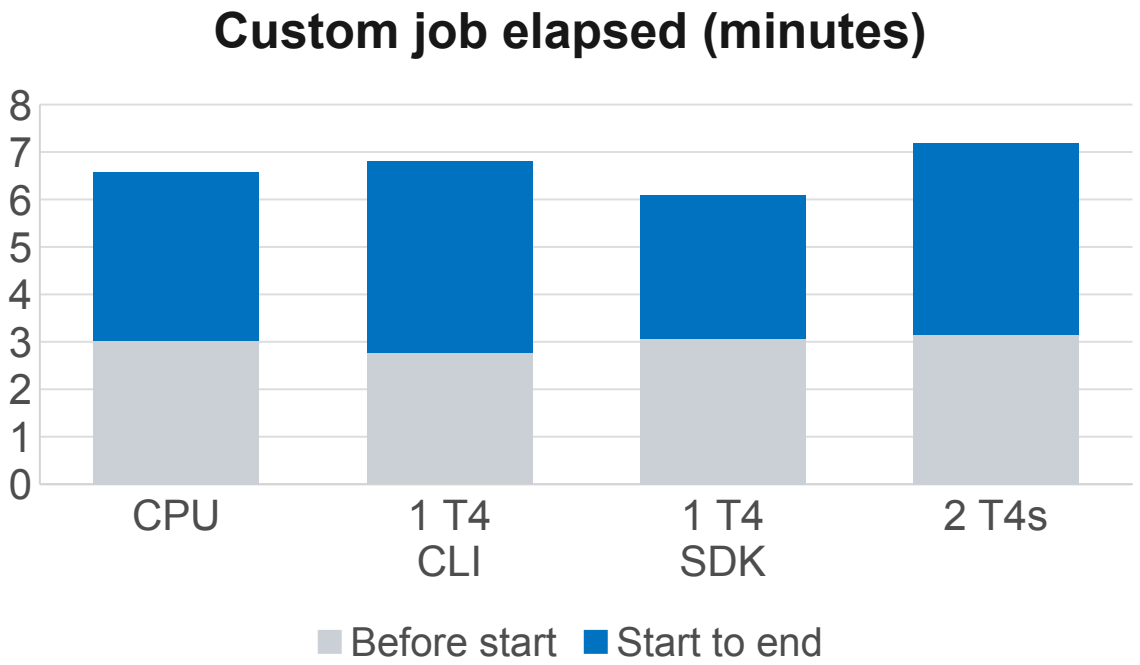
CPU run One T4 CLI One T4 SDK Two T4s

936 training / 104 validation images; n1-standard-4 for all routes. CLI/SDK weights match.
116 test images unscored. One epoch checks the workflow; DDP details come later.

Cheese: training time and cloud-job time

model.fit includes training and validation. The job also includes setup.





Same host: 4 vCPUs. Same global batch: 32. One recorded run per route.

Start to end includes model.fit. SDK pipeline creation is outside this chart.
These durations do not establish a scaling speedup or an actual billing charge.

Cheese: accepted W&B runs

Follow the four accepted runs from the controlled comparison

Accepted runs in W&B

CPU

One T4, CLI

One T4, Python SDK

Two T4s, synchronized copies

W&B links require project access.
This tutorial did not score its final test split or export a reloadable model.

Custom training image: uv and PyTorch

Conceptual option: build a separate, locked training environment into an image.

```
ARG UV_IMAGE
ARG BASE_IMAGE
FROM ${UV_IMAGE} AS uv_tool
FROM ${BASE_IMAGE} AS runtime
COPY --from=uv_tool /uv /bin/uv
WORKDIR /app
ENV UV_PYTHON_DOWNLOADS=0
ENV UV_PROJECT_ENVIRONMENT=/app/.venv
COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-dev --no-install-project
COPY trainer/ trainer/
ENTRYPOINT ["/app/.venv/bin/python", "-m", "trainer.task"]
```

Training pyproject.toml pins torch and its CUDA index. Supply reviewed image digests as arguments. Two FROM stages copy uv during the build. Exclude the Mac .venv and credentials from the context.

Custom image: build, push, then submit

Future option. The measured cheese runs used the earlier TensorFlow route.

```
docker buildx build --platform=linux/amd64 --push \
  --build-arg BASE_IMAGE="$BASE_IMAGE" --build-arg UV_IMAGE="$UV_IMAGE" \
  -t "$IMAGE_TAG" .
```

After the push, Vertex pulls the finished image by digest

```
# aip initialized for project / region, as before
job = aip.CustomContainerTrainingJob(
    display_name="pytorch-demo", container_uri=IMAGE_DIGEST_URI)
job.run(
    replica_count=1, machine_type="n1-standard-4",
    accelerator_type="NVIDIA_TESLA_T4", accelerator_count=1,
    service_account=TRAINER_SA, args=["--epochs=1"])
```

IMAGE_TAG is the image address + tag in Artifact Registry, Google's container image store.
IMAGE_DIGEST_URI pins the uploaded bytes by hash. The worker needs pull access and GPU quota.

Outline

1. Models we are looking at
2. Testing model behavior
3. Experiment tracking with W&B
4. Vertex AI / Agent Platform training
5. **Parallel training**

Why use more than one GPU?

Two common reasons: train on more examples, or fit a larger model

Setup	What changes?	TensorFlow name
One machine, one GPU	One device does the work	OneDeviceStrategy
One machine, several GPUs	Model copies share the examples	MirroredStrategy
Several machines, several GPUs	Workers also talk over a network	MultiWorkerMirroredStrategy

Data parallelism gives model copies different examples. Model parallelism divides model computation.
We will follow one training step before comparing these choices.

One training step

Before adding GPUs, follow what one GPU does

1. Predict

Use current weights to answer examples.

2. Measure error

Compare predictions with correct labels.

3. Find gradients

Find how the loss changes with weights.

4. Update weights

Use gradients to change the weights.

Tiny example: weight 10, gradient 2, learning rate 0.1

Gradient descent gives $10 - 0.1 \times 2 = 9.8$.

Weights are the model's learned numbers. Loss measures prediction error.

Workers, GPUs, and machines

A PyTorch worker is a running copy of the training program

One machine (also called a node)

GPU 0
Worker ID 0

GPU 1
Worker ID 1

PyTorch calls the worker ID its rank. Here, ranks are 0 and 1.

A tensor is an array of numbers. Workers exchange numbers to coordinate their updates.

DDP: splitting one global batch

Distributed Data Parallel lets full model copies work on different examples

Global batch: A, B, C, D

GPU 0: full model copy
Examples A and B

GPU 1: full model copy
Examples C and D

Each computes forward + backward. DDP averages the gradients.

The averages go back to both ranks. Both apply the same update.

The input pipeline assigns the examples. Here, 2 per GPU × 2 GPUs = global batch 4.
Both ranks receive the same averaged gradients, so their model copies stay synchronized.

TensorFlow and PyTorch: who runs the code?

Python executes on CPUs. Framework operations send tensor work to GPUs.

TensorFlow: one process, two GPUs

```
python train_tf.py
```

MirroredStrategy creates replicas.
model.fit runs their training steps
and synchronizes gradients.
A custom loop uses strategy.run
to call your step on each replica.

PyTorch: one process per GPU

```
torchrun --standalone \
  --nproc-per-node=2 train.py
```

torchrun starts train.py twice.
Each process selects its GPU
and reads its own data slice.
DDP or FSDP coordinates sharing
by exchanging model data.

On Vertex or Modal, the worker container runs the launch command. The platform supplies GPUs.
The trainer and its framework determine how those GPUs execute the training program.

TensorFlow: two GPUs, one machine

MirroredStrategy: synchronous data parallelism (Keras 3.13 illustration)

1. Split each global batch across GPUs

```
s = tf.distribute.MirroredStrategy(
    devices=["/gpu:0", "/gpu:1"])
batches = dataset.batch(64,
    drop_remainder=True)  # global batch
dist_ds = (
    s.experimental_distribute_dataset(
        batches))
# 32 different examples per GPU
```

2. Build and compile once in scope

```
loss_fn = SparseCategoricalCrossentropy(
    from_logits=True)
with s.scope():
    model = build_model()  # logits
    model.compile(optimizer=SGD(.1),
                  loss=loss_fn)
model.fit(dist_ds, epochs=1)
```

One compile call. fit runs replica steps and combines their gradients.

One process, two full replicas. Global batch 64 = 32 examples per GPU in both frameworks.
Cheese uses Keras 3.5 with an explicit global-mean objective. Here, drop_remainder drops the tail.

TensorFlow: one compile call, two replicas

scope() sets the strategy for state creation. Python enters it once.

One CPU Python process

```
# Reuse s and dist_ds from the previous slide
with s.scope():
    model = build_model()
    model.compile(optimizer=SGD(.1),
                  loss=loss_fn)

model.fit(dist_ds, epochs=1)
```

What each call does

build_model: create mirrored weights.
compile: configure optimizer and loss.
fit: run steps on both replicas.

Inside fit: each GPU computes forward/backward on its own batch slice.

GPU 0: state + its batch slice

GPU 1: state + its batch slice

fit uses the model's captured strategy, so it can appear outside the with block.
Optimizer state may be created lazily by fit. Moving fit alone cannot mirror existing weights.

PyTorch: one CPU process per GPU

torchrun starts train.py twice. Each copy receives its own GPU number.

```
torchrun --standalone --nproc-per-node=2 train.py
```

CPU process 0
LOCAL_RANK = 0
Uses GPU 0

CPU process 1
LOCAL_RANK = 1
Uses GPU 1

Python runs on the CPU. Each worker sends tensor operations to its GPU.

LOCAL_RANK chooses a GPU on this machine. Global rank identifies a worker across machines.
The next slide is train.py. Both processes execute it, and DDP synchronizes their gradients.

PyTorch: two GPUs, one machine

Both train.py processes run this code, selecting different GPUs

1. Copy the model and split the data

```
gpu = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(gpu)
dist.init_process_group("nccl")
model = DDP(net.to(gpu),
            device_ids=[gpu])
sampler = DistributedSampler(
    data, drop_last=True)
loader = DataLoader(data, batch_size=32,
    sampler=sampler, drop_last=True)
opt = SGD(model.parameters(), lr=.1)
```

2. Train each local batch

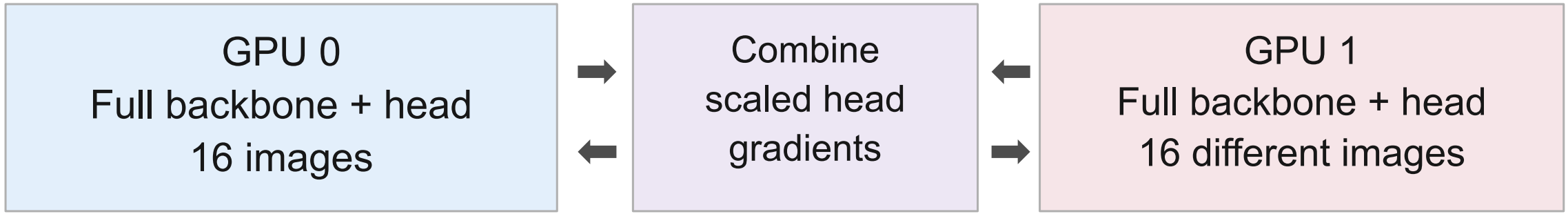
```
for epoch in range(epochs):
    sampler.set_epoch(epoch)
    for x, y in loader:
        x, y = x.to(gpu), y.to(gpu)
        opt.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        opt.step()
```

loss.backward() averages gradients. opt.step() updates each full copy.

Model and batch tensors move to the selected GPU with .to(gpu).
Generic example drops tails for global 64. Actual babykev uses its own batch partitioner.

Cheese on two T4s: the DDP idea in TensorFlow

One VM, two T4s, one TensorFlow process, two model replicas



Same synchronous data-parallel idea as DDP.

Each replica updates its head with the same combined gradient.

Loss contributions use the actual global count before SUM. Full weights stay on each GPU.
Controlled: global 32 = 16 + 16, tail 8 = 4 + 4, 30 updates. Original tutorial: global 64 = 32 + 32.

What occupies GPU memory?

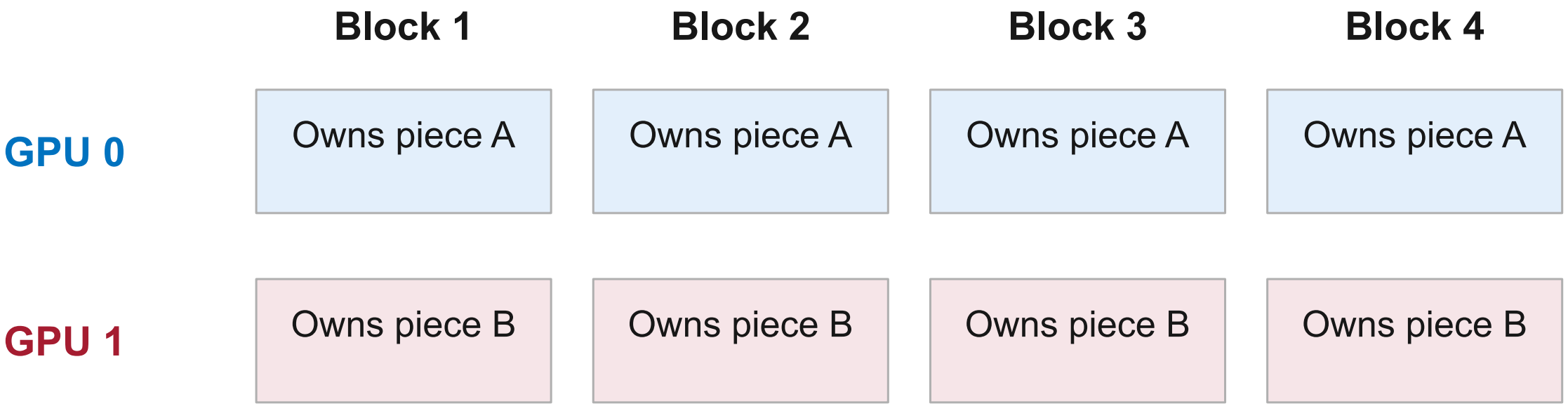
The weights are only part of the training memory

What is stored?	What is it for?
Weights	The model numbers used to predict
Gradients	How loss changes with each trainable weight
Optimizer state	History used to choose the next update
Activations	Intermediate results saved for backward

LoRA trains small adapters and the task head. Their gradients and optimizer state are smaller.
The frozen backbone still occupies memory. Actual peak memory must be measured.

FSDP: each GPU stores pieces of every block

Fully Sharded Data Parallel divides model storage across GPUs.

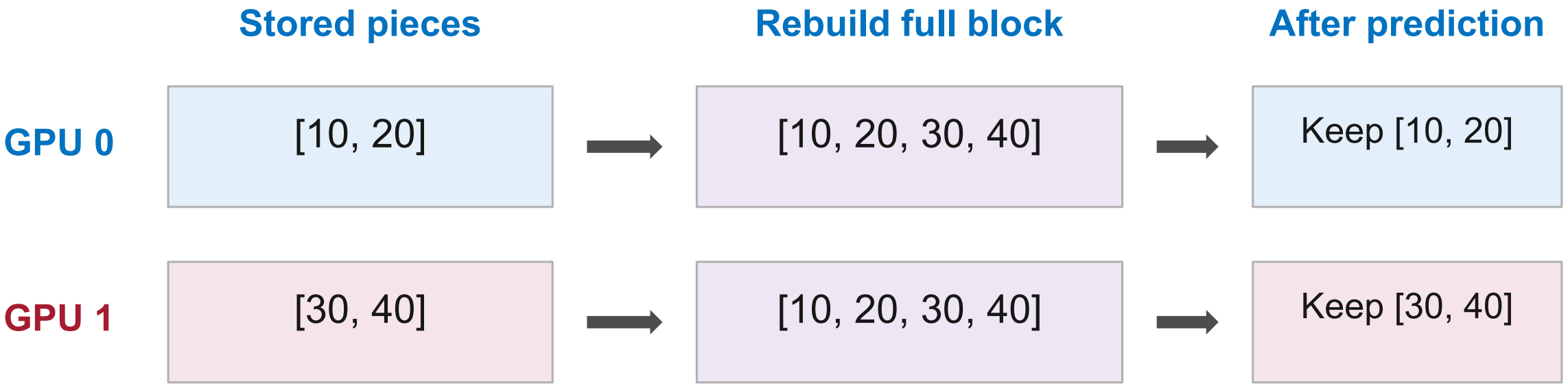


Each GPU reads different examples and runs blocks 1 → 2 → 3 → 4.

For trainable weights, each GPU also stores its pieces of the gradients and optimizer history.
A full block is rebuilt temporarily when that GPU needs to compute it.

FSDP forward: rebuild one block, then free it

For EACH block, both GPUs receive the weight pieces they are missing.

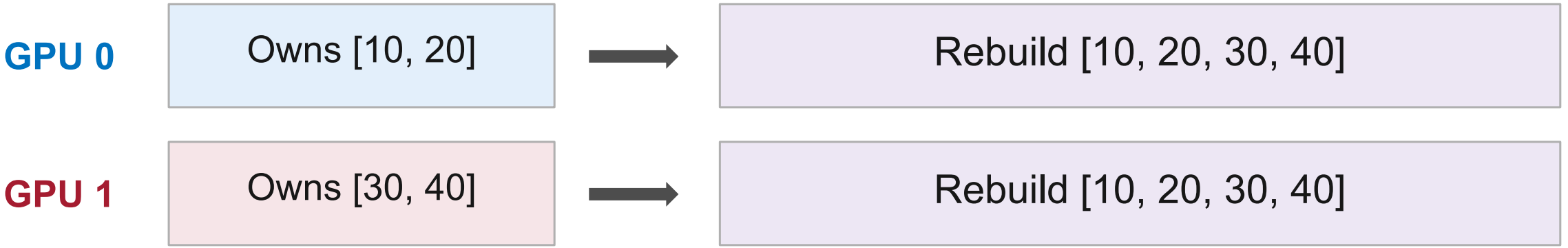


Predict on each GPU's own examples. Save activations; free the full weights.

Continue with the next block. Each GPU keeps only its owned weights between blocks.
The saved activations remain for backward. Weight values have not changed yet.

FSDP backward: rebuild each block AGAIN

Start at the last block. Both GPUs need its full, unchanged weights again.



Use saved activations + incoming gradient to calculate this block's gradients.

Average both GPUs' contributions for each owned weight piece:

GPU 0 keeps gradients for piece A	GPU 1 keeps gradients for piece B

Send contributions to their owner; free full weights. Repeat for blocks 4 → 3 → 2 → 1.
Only after ALL blocks finish backward: update the owned trainable weights.

FSDP: memory while one block computes

Four blocks × 4 GiB = 16 GiB of model weights. Two GPUs.

On one GPU	Owned pieces	Full block buffer	Weight subtotal
Between blocks	8 GiB	0 GiB	8 GiB
While a block computes	8 GiB	4 GiB	12 GiB
After releasing it	8 GiB	0 GiB	8 GiB

DDP keeps all 16 GiB of weights on each GPU throughout training.

Weight accounting only; no overlapping blocks. Add activations, training state and other buffers.
Peak memory needs measurement. Split storage block by block to avoid rebuilding the whole model.

PyTorch FSDP: split storage, keep the training loop

Every torchrun worker runs this file, using its own GPU and data slice

1. Each rank keeps model pieces

```
# Same torchrun/device setup as DDP
# Start with identical model weights
for block in model.layers:
    fully_shard(block,
        reshard_after_forward=True)
fully_shard(model,
    reshard_after_forward=True)
opt = AdamW(model.parameters())
```

2. Each rank trains its slice

```
for x, y in local_loader:
    x, y = x.to(gpu), y.to(gpu)
    opt.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    opt.step()
```

Inside loss.backward(), for each block:
rebuild weights → compute gradients;
free full weights → combine gradient pieces.
opt.step() then updates this GPU's pieces.

fully_shard adds automatic actions around each block's forward and backward computation.
True frees full weights after forward. Backward rebuilds them; no extra call in your loop.

DDP and FSDP: same data split, different storage

Both GPUs process different examples and compute every layer

At this point	DDP: copies	FSDP: pieces
Between computations	Full weights + training state	Owned weight and state pieces
Before a layer computes	Weights already present	Rebuild the layer’s full weights
During backward	Average gradients for both GPUs	Combine gradients at each owner
Optimizer step	Update the full trainable copy	Update the owned trainable piece

DDP is useful when the full training copy fits. FSDP reduces persistent storage per GPU.
Temporary full layers and activations still use memory. Speed and peak memory need measurement.

GPU memory and shared host memory

Two GPUs in one machine still have separate device memories

Host RAM: CPU memory available to both worker processes

PCIe or NVLink



GPU 0
Its own device memory

GPU 1
Its own device memory

Without direct peer access, transfers can stage through host RAM.

Shared host RAM does not combine the GPU memories into one pool. FSDP manages ownership.
CUDA “shared memory” inside a GPU kernel is a separate concept.

Links between GPUs

GPUs exchange weights and gradients over different kinds of connections

Link	What connects the GPUs?	What determines the route?
PCIe	The host's I/O fabric. Peer access can avoid a host RAM copy.	GPU/driver support, switches and host topology
NVLink / NVSwitch	Direct GPU links or a switched GPU communication fabric	Which GPUs have links, link generation and count
Between machines	Network cards using InfiniBand or Ethernet, with direct GPU access or host staging	Network setup, direct-access support and placement

DDP exchanges gradients. FSDP also exchanges weight pieces, often many times per update. Frequent transfers make the links important. GPU model alone does not specify the connections.

Colab, Vertex AI and Modal: where the program runs

The platform starts compute. Our program defines the training algorithm.

Platform	How we launch	Resource / storage choice
Colab	Open and run a notebook	Runtime allocation varies. Save outputs outside the temporary VM.
Vertex AI / Agent Platform	Submit a packaged training job with CLI or Python SDK	Choose region, VM and GPUs. Use Cloud Storage for artifacts.
Modal	Call a remote Python function in a configured container	Declare GPU type/count. Use persistent Volumes or storage.

Colab suits interactive class work. Vertex and Modal also support submitted training runs.
Quota, capacity and persistence differ. DDP and FSDP are choices inside the training program.

babykev on Modal: the launcher starts each rank

train() runs once on a CPU. Its launcher starts one CPU process per GPU.

```
execute(config_path, data, out, Path("/runs"), resume_path)
```

Inside execute(): launch the module below, using this container's Python.

```
python -m torch.distributed.run --nproc_per_node=2 \
  --master_addr=127.0.0.1 --master_port=<port> --max_restarts=0 \
  -m parallel_kev.launch --worker --config <recipe> \
  --data <data> --out <run> --output-root /runs
```

DDP recipe: global batch 8, microbatch 1 (one request per forward).
Each rank sums four microbatch gradients for one optimizer step.

CPU rank 0 selects GPU 0 Complete KEV replica Data slots: 0, 2, 4, 6	CPU rank 1 selects GPU 1 Complete KEV replica Data slots: 1, 3, 5, 7

Angle brackets are values supplied by execute(); it chooses a free local port. Resume adds --resume.
The remote train() function calls this execute() helper inside the GPU container.

Actual cheese on Vertex: TensorFlow manages replicas

The submitted TensorFlow module creates the replicas and dispatches GPU work

1. Submit

cli-multi-gpu.sh selects mode multi in our helper.
The job names trainer.task_multi_gpu, one VM and two T4s.

2. Vertex worker

Run the existing TensorFlow image, install the source tar, then execute the specified trainer module.

3. Training Python

```
strategy = tf.distribute.MirroredStrategy()
# Inside strategy.scope(): build and compile the model
model.fit(train_ds, ...)  # condensed control flow
```

4. Each update

Full batch: 32 (16 per GPU). Tail: 8 (4 per GPU).
Global-mean loss makes summed gradients correct.

The TensorFlow application drives the replicas. This executed cheese route uses no torchrun.
Both GPUs compute the frozen backbone and trainable head, using the DDP idea.

Saving enough to continue training

A saved model can predict. Resuming training needs more state.

Save	Why it matters
Weights	Keep what the model has learned
Optimizer state	Keep the history that determines the next update
Data position and counters	Know which examples and step come next
Random state for every worker	Continue sampling and dropout consistently
Distributed pieces and metadata	Reconstruct shards and local/outer optimizer state

Compare an interrupted run with an uninterrupted run to check recovery.
Changing the number of workers can change the learning path.

Local recovery checkpoints and Hugging Face exports

Storage supports both resuming training and loading a model for prediction

Resume: local disk or Modal Volume

```
step-000008/
  rank-00000.pt
  rank-00001.pt
  manifest.json
```

Rank files + manifest preserve weights, optimizer, per-worker random state and the next data position.

Hub: versioned inference export

```
adapter.safetensors
head.safetensors
checkpoint.json
```

LoRA weights + pointer head + recipe.
Base Qwen weights load separately.
Model card, license and hashes travel too.

babykev-3b-cheese (private model repo)

Pin the Hub export and its base revision. Private repositories require access.
This inference export omits optimizer/rank state. Keep the complete checkpoint to resume.

Inside a local training checkpoint

A tiny CPU example shows the same file structure as the course checkpoints

Read the local file with PyTorch

```
import torch
p = "work/checkpoint-demo/"
p += "step-000001/rank-00000.pt"
s = torch.load(p, map_location="cpu",
    weights_only=True)
s["weights"]["head.weight"]  # rounded:
[[ 0.0990, 0.1990, 0.2990],
 [-0.0990, 0.0010, 0.1010]]
```

What the saved dictionary contains

weights: head.weight (2 × 3, FP32)
optimizer: AdamW step 1 + history
torch_rng, python_rng: random states
cuda_rng: None (CPU example)
cursor: window 1 (next batch), step 1
epoch 0, records 2, tokens 0

manifest.json (SHA-256 prefixes shortened)

config: toy recipe, world_size=1 | data: toy-data.json → ff56d775…
files: rank-00000.pt → 8c987964… | provenance: source + runtime

Toy dimensions/values; not a KEV checkpoint. Frozen weights stay in pinned frozen-base.pt.
Two ranks also need rank-00001.pt and matching recipe, data, source/runtime and frozen base.

Choosing a method

First identify the problem you need to solve

Problem	Method to investigate
Model fits; process examples together	DDP: model copies, shared gradients
Copies use too much training memory	FSDP: split stored model state
A layer or stack needs several GPUs	Tensor or pipeline parallelism
Frequent sharing is expensive	Local SGD or DiLoCo; check learning quality

Compare the same task and data budget. Record GPU count, batch, memory, speed, and quality. Keep LoRA and full parameter experiments separate. These choices need measurements.

GPU options on Modal and Vertex AI

Checked October 19, 2026

Modal GPU guide

Vertex GPU / machine table

GPU	GB / GPU	Modal	Vertex AI	GPU link capability*
T4 / L4	16 / 24	Listed	Listed	PCIe 3 / PCIe 4
A10 / L40S	24 / 48	Listed	Not listed	PCIe 4 (L40S: no NVLink)
A100	40 or 80	Listed	Listed	NVLink-capable variants
RTX PRO 6000	96	Listed	Listed	PCIe 5
H100	80	Listed	Listed	NVLink (SXM on Modal)
H200	141	Listed	Listed	NVLink (SXM on Modal)
B200	180	Listed	Listed	NVLink / NVSwitch systems
B300	288	Listed	Not listed	NVLink / NVSwitch systems
GB200	186	Not listed	Listed	NVLink fabric + Arm CPU C2C

SXM = a GPU module on a server board. NVLink = a link. PCIe cards can also support NVLink.
*Capability differs from allocated links. Check topology, GPU count and peer access.
Legacy Vertex: P4/V100. P100 retired Sept 15, 2026. GB200 C2C links CPU and GPUs.



THANK YOU